

"Você consegue, sei lá"

Caroline Coutinho

Teoria: Apache Spark um guia dos princípios

Uma imersão nos fundamentos da computação distribuída e o poder do Apache Spark para Big Data.



Por que Spark? Escalabilidade em Foco



Scaling UP (Vertical)

Aumentar o poder de uma única máquina (mais RAM/CPU).

Limitações físicas rapidamente atingidas, impedindo o processamento de grandes volumes de dados.



Scaling OUT (Horizontal)

Distribuir a carga de trabalho entre múltiplas máquinas conectadas em rede (um cluster). O Spark adota essa abordagem, "dividindo para conquistar" problemas complexos.

Spark vs. MapReduce: A Evolução do Processamento

O Legado do Hadoop MapReduce

Um modelo de processamento que lê dados do disco, processa e depois escreve de volta no disco. Esse fluxo constante de I/O o torna intrinsecamente lento e ineficiente para Big Data.

- **Leitura de Disco**
- **Processamento**
- **Escrita em Disco**

A Revolução do Apache Spark

O Spark otimiza o processamento mantendo os dados na memória (RAM) sempre que possível. Isso resulta em uma velocidade até 100x maior em cargas de trabalho iterativas e interativas.

- **Processamento In-Memory:**
- **I/O Reduzido**
- **Cache**

Custo de Rede: Trafegando Código vs. Dados

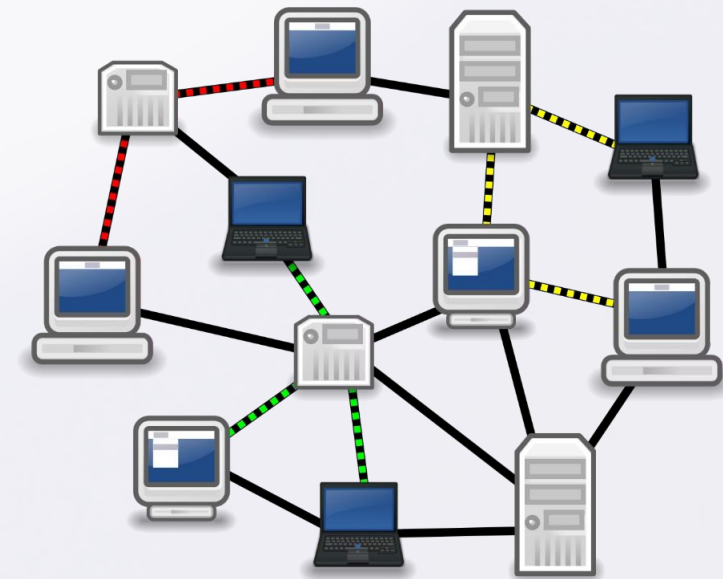
Em ambientes distribuídos, o tráfego de dados pela rede é um dos maiores gargalos. O Spark adota uma estratégia inteligente para minimizar esse custo:

Trafegar Código (Mais Barato)

É mais eficiente enviar a função de processamento (o "código") para os nós onde os dados já residem. Dessa forma, apenas os resultados reduzidos ou agregados precisam ser enviados de volta, otimizando o uso da rede.

Trafegar Dados (Mais Caro)

Mover grandes volumes de dados de um nó para outro para serem processados é custoso em termos de tempo e recursos de rede, impactando diretamente a performance.



Anatomia do Cluster Spark: Mestre e Operários



Driver (O Mestre/Gerente)

Traduz o código do usuário em tarefas executáveis, gerencia a execução e mantém o `SparkContext`. Não processa dados pesados, delegando essa função aos Executors.



Executors (Os Operários)

Processos JVM que rodam nos nós (workers), responsáveis por executar as tarefas distribuídas e armazenar dados em cache. Cada Core de um Executor pode processar uma tarefa por vez.



Cluster Manager

O responsável por alocar recursos (RAM/CPU) para a aplicação Spark. Exemplos: Standalone, YARN, Kubernetes, Mesos.

O Modelo de Execução: Lazy Evaluation & DAG

Um dos conceitos mais cruciais para entender o desempenho do Spark.

01

Lazy Evaluation (Avaliação Preguiçosa)

O Spark não executa operações imediatamente. Ele apenas registra as "transformações" (`map`, `filter`) até que uma "ação" (`count`, `write`, `show`) seja invocada. Isso permite otimizações globais.

02

DAG (Directed Acyclic Graph)

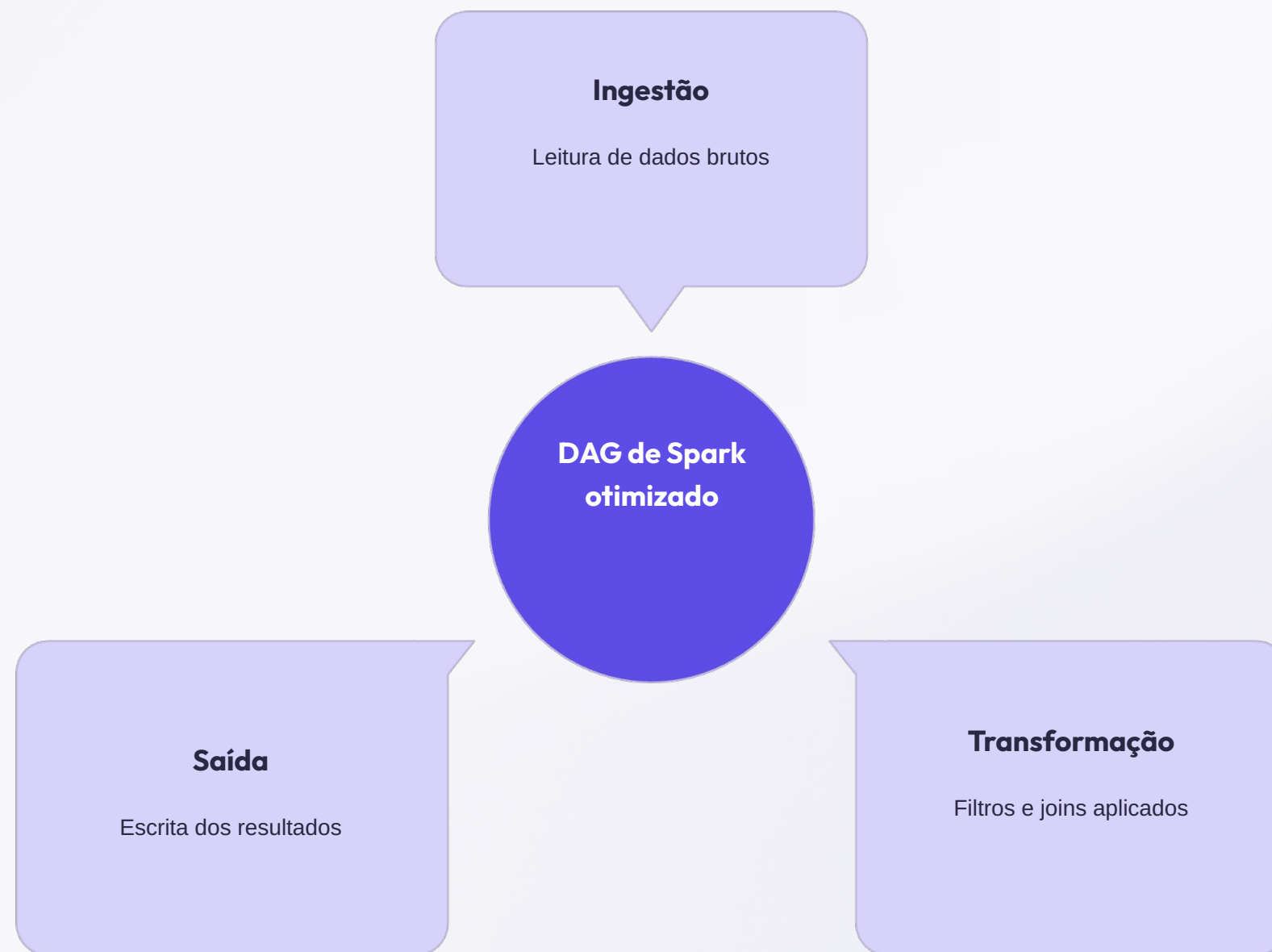
Quando uma ação é disparada, o Spark constrói um "Plano de Voo" (DAG) das operações. Ele otimiza esse grafo, combinando e reordenando etapas para a execução mais eficiente.

03

Hierarquia de Execução

- **Application**
- **Job**
- **Stage**
- **Task**

Entendendo o DAG: Otimização Inteligente



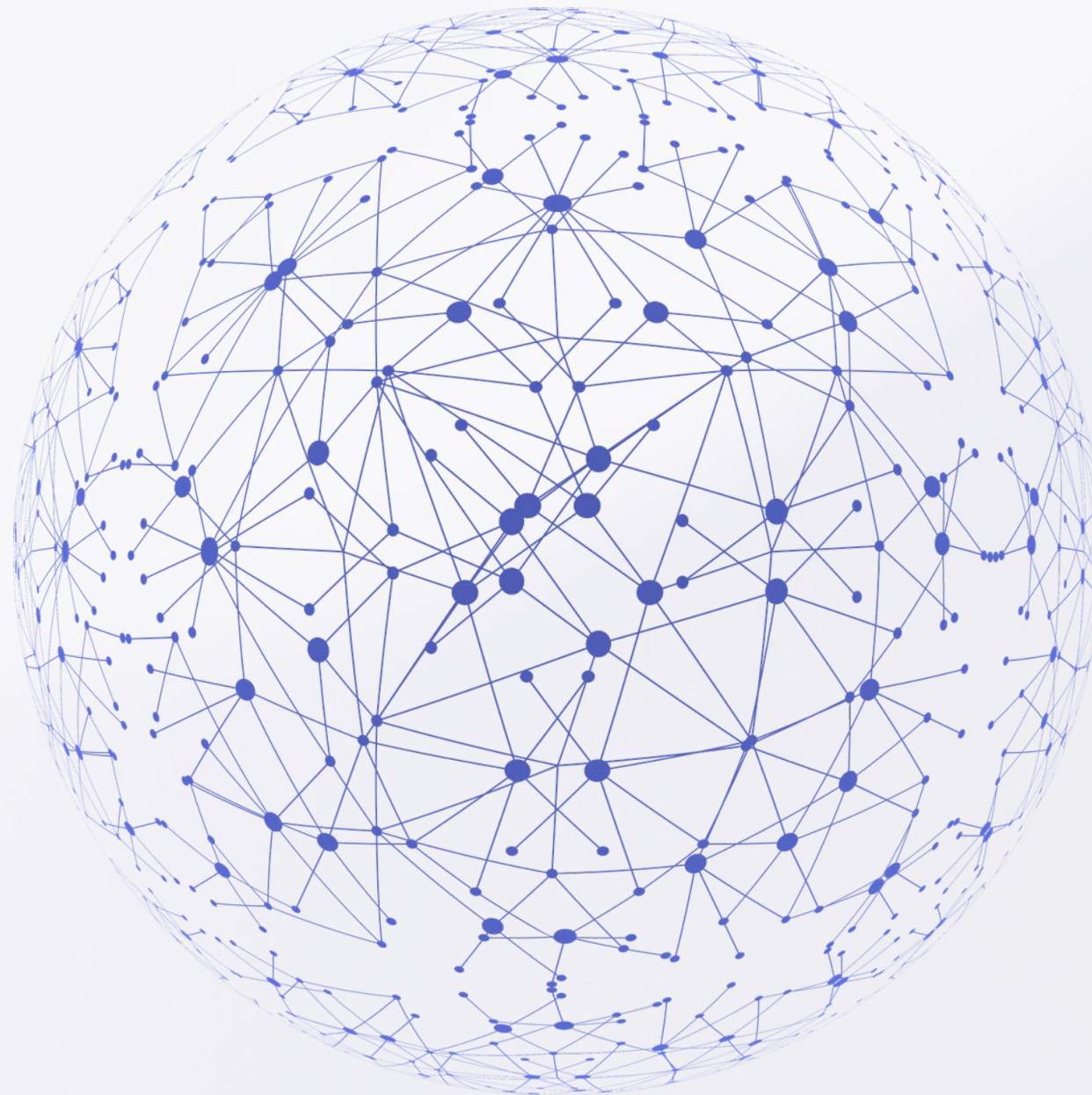
O DAG permite que o Spark visualize todas as operações a serem realizadas, otimizando o plano de execução antes de qualquer computação real. Isso minimiza o tempo de processamento e o uso de recursos.

RDDs: Os Blocos Fundamentais do Spark

RDD (Resilient Distributed Dataset)

A "linguagem de montagem" do Spark. Um RDD é uma coleção de elementos tolerante a falhas e distribuída entre nós. Possui características essenciais:

- **Imutabilidade**
- **Particionado**
- **Tolerante a Falhas**



Linhagem (Lineage) e Recuperação de Falhas

Linhagem: O "DNA" do RDD

O Spark mantém um registro das transformações que foram aplicadas para criar um RDD. Essa "linhagem" é essencial para a tolerância a falhas.

- **Sem Backup Físico:**
- **Recomputação Eficiente:**
- **Resiliência:**

DataFrames & Datasets: Abstrações Modernas

A evolução da API do Spark para facilitar o trabalho com dados estruturados.

DataFrames

Representam dados como uma coleção distribuída de linhas nomeadas, semelhante a tabelas de banco de dados. Oferecem otimizações significativas através do Projeto Tungsten e Catalyst Optimizer.



Datasets

Estendem os DataFrames com verificação de tipo em tempo de compilação, oferecendo benefícios de performance e segurança para linguagens como Scala e Java. Para Python, DataFrames são a principal abstração.

DataFrames são mais rápidos que RDDs em Python devido à sua interoperabilidade otimizada com a JVM do Spark e a camada de otimização de execução, superando as limitações da serialização Python.